

Using a Machine Learning Model to Make Predictions

Machine learning is basically a subset of artificial intelligence that uses previously existing data to make a prediction on new data. Of course, all of us know this by now! This article demonstrates how a machine learning model developed in Python can be used as a part of a Java code to make predictions.



This article assumes you are familiar with the basic development skills and understanding of machine learning. We will start with training our model, and then make a machine learning model in Python.

I am taking the example of a flood prediction model. First, import the following libraries:

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
```

Once we have successfully imported the libraries, we need to take in the data

sets, as shown in the code below. To predict floods, I am using the river level data set.

```
from google.colab import files
uploaded = files.upload()
for fn in uploaded.keys(): print('User
uploaded file "{name}" with length
{length} bytes'.format(
name=fn, length=len(uploaded[fn])))
Choose files No file chosen
```

The upload widget is only available when the cell has been executed in the current browser session. Please rerun this cell to enable *Saving Hoppers Crossing-Hourly-River-Level.csv* to *Hoppers Crossing-Hourly-River-Level.csv* User uploaded file "*Hoppers Crossing-Hourly-River-Level.csv*" with length 2207036 bytes.

Once this is done, we can train our model using the *sklearn* library. For this, we first need to import the library and the algorithm model, as shown in Figure 1.

```
from sklearn.linear_model import LinearRegression
regressor = LinearRegression()
regressor.fit(X_train, y_train)

LinearRegression(copy_X=True, fit_intercept=True, n_jobs=None, normalize=False)
```

Figure 1: Training the model

```
predicted_riverlevel = regressor.predict(a)
print(predicted_riverlevel)
if (predicted_riverlevel > 1.5):
    print("FLOOD")
else:
    print("No FLOOD")

[[1.64726206]]
FLOOD
```

Figure 2: Making predictions

```
from sklearn.linear_model import
LinearRegression
regressor = LinearRegression()
regressor.fit(X_train, y_train)
```

Once that is done we have trained our model, and it's now ready to make predictions, as shown in Figure 2.

Using ML model in Java

What we need to do now is to convert the ML model into a model that can be used by a Java program. There is a library called *sklearn2pmml* that helps us do this:

Install the library,
`pip install sklearn2pmml`

Once the library is installed we can convert our already trained model, as shown below:

```
sklearn2pmml(pipeline, 'model.pmml',
with_repr = True)
```

This is it! We can now use the generated *model.pmml* file in our Java code to make predictions. Do try it out! 

 By: Jishnu Saurav Mittapalli

The author is a full stack Web developer, interested in modern technologies like artificial intelligence and machine learning. He is currently working in the field of natural language processing.